

Locality-Sensitive Hashing

Approximate Nearest Neighbor Search at Scale

Hoa Vu
CS 663

SDSU

April 22, 2026

Nearest Neighbor Search

Input: A dataset P of n points in $\mathbb{R}^d$, a query point q .

Goal: Find the point in P closest to q under some distance metric.

Naive: scan all n points $\Rightarrow O(nd)$ per query. Too slow for large n , large d .

Approximate Nearest Neighbor

Exact nearest neighbor is hard in high dimensions (curse of dimensionality).

Relaxation: Find a point $p^* \in P$ such that

$$d(q, p^*) \leq (1 + \varepsilon) d(q, p_{\text{true}})$$

Much faster in practice, good enough for most applications.

Locality-Sensitive Hashing (LSH)

Idea: Hash points so that *nearby points collide* more often than far-apart points.

$\Pr[h(p) = h(q)]$ is **high** when p, q are close

$\Pr[h(p) = h(q)]$ is **low** when p, q are far

Candidate pairs = same bucket. Only check those.

LSH Family: Definition

A family $\mathcal{H}$ is (r_1, r_2, p_1, p_2) -**sensitive** if for any p, q :

$$\blacktriangleright \text{ If } d(p, q) \leq r_1 \quad \Rightarrow \quad \Pr[h(p) = h(q)] \geq p_1$$

$$\blacktriangleright \text{ If } d(p, q) \geq r_2 \quad \Rightarrow \quad \Pr[h(p) = h(q)] \leq p_2$$

Useful when $r_1 < r_2$ and $p_1 > p_2$.

MinHash is LSH

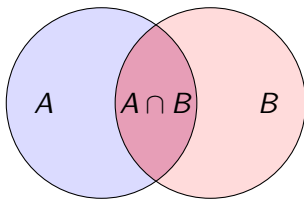
Recall: for sets A, B ,

$$\Pr[\text{minhash}(A) = \text{minhash}(B)] = J(A, B)$$

So a single MinHash function is a (s_1, s_2, s_1, s_2) -sensitive family under Jaccard distance $d = 1 - J$.

One hash alone gives weak separation. We amplify with **banding**.

Recall: Jaccard Similarity



$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Example: $A = \{1, 2, 3, 4\}$, $B = \{3, 4, 5, 6\}$

$$|A \cap B| = 2, \quad |A \cup B| = 6 \quad \Rightarrow \quad J(A, B) = \frac{2}{6} = \frac{1}{3}$$

$$d(A, B) = 1 - J(A, B) = \frac{2}{3}.$$

Recall: MinHash

Given a set A and a hash function h on elements, define:

$$\text{minhash}(A) = \min_{x \in A} h(x)$$

Example: $A = \{a, b, c, d, e\}$ with hash values:

element	$h(x)$
a	7
b	3
c	9
d	5
e	11

→ $\text{minhash}(A) = 3$

Step 1: The Key Identity

For any two sets A and B , a random MinHash function h satisfies:

$$\Pr[h(A) = h(B)] = J(A, B)$$

Equivalently, using Jaccard distance $d = 1 - J$:

$$\Pr[h(A) = h(B)] = 1 - d(A, B)$$

So the collision probability *equals* the Jaccard similarity, no approximation needed.

Step 2: Close Pairs

Recall the LSH goal: close pairs should collide with *high* probability.

Suppose $d(A, B) \leq 1 - s_1$, i.e. the sets are similar: $J(A, B) \geq s_1$.

Then:

$$\Pr[h(A) = h(B)] = J(A, B) \geq s_1$$

The higher the similarity, the higher the collision probability.

Step 3: Far Pairs

Recall the LSH goal: far pairs should collide with *low* probability.

Suppose $d(A, B) \geq 1 - s_2$, i.e. the sets are dissimilar: $J(A, B) \leq s_2$.

Then:

$$\Pr[h(A) = h(B)] = J(A, B) \leq s_2$$

The lower the similarity, the lower the collision probability.

Conclusion: MinHash is LSH

Putting it together: MinHash is a

$(1 - s_1, 1 - s_2, s_1, s_2)$ -sensitive family

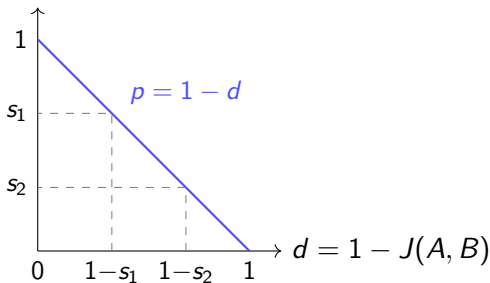
under Jaccard distance, for any $0 \leq s_2 < s_1 \leq 1$.

When we parameterise by similarity thresholds directly, this is written (s_1, s_2, s_1, s_2) .

Limitation: a single hash gives only weak separation ($p_1 = s_1, p_2 = s_2$ are close). We need amplification.

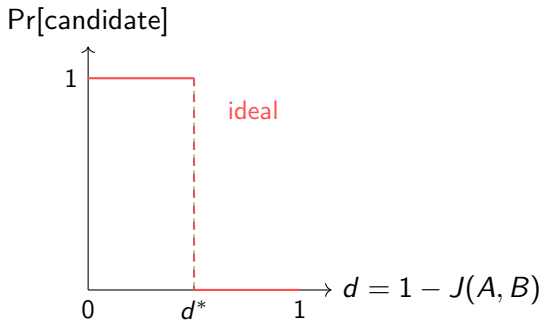
Single MinHash: Collision Probability vs. Distance

$$p = \Pr[h(A) = h(B)]$$



A linear relationship; no amplification yet.

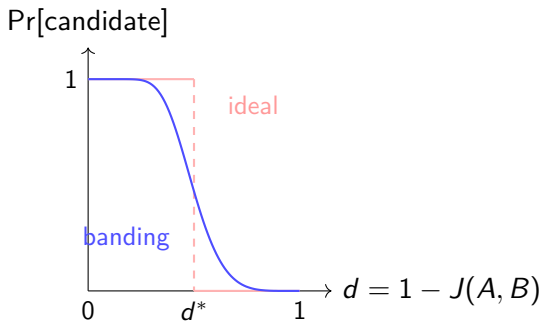
What We Ideally Want



A perfect step at threshold d^* : all close pairs found, no far pairs flagged.

Banding approximates this with the S-curve.

Banding Gives an S-Curve



The AND-OR Construction

Take $k = b \times r$ hash functions. Arrange into b **bands** of r rows each.

	item A	item B	
band 1	$h_1(A)$	$h_1(B)$	match
	$h_2(A)$	$h_2(B)$	
	$h_3(A)$	$h_3(B)$	
band 2	$h_4(A)$	$h_4(B)$	no match
	$h_5(A)$	$h_5(B)$	
	$h_6(A)$	$h_6(B)$	
band 3	$h_7(A)$	$h_7(B)$	match
	$h_8(A)$	$h_8(B)$	
	$h_9(A)$	$h_9(B)$	

matches: if *all* r rows agree. **no match:** if *any* band does not match.

Candidate Probability

A pair (A, B) is a **candidate** if it shares the same bucket in at least one band.

Only candidates are checked for exact similarity, avoiding $O(n^2)$ comparisons.

Let $p = J(A, B)$. How likely is a pair to become a candidate?

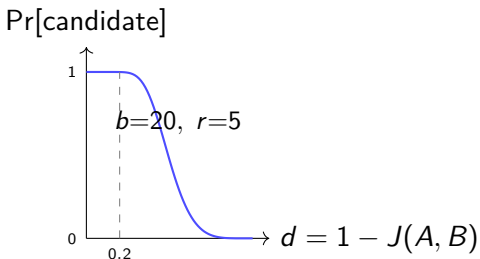
- ▶ p^r prob. all r rows in a band match (**AND**)
- ▶ $(1 - p^r)^b$ prob. *no* band matches (**OR**)
- ▶ $1 - (1 - p^r)^b$ prob. at least one band matches

Using $1 - x \approx e^{-x}$:

$$(1 - p^r)^b \approx e^{-bp^r} \Rightarrow \Pr[\text{candidate}] \approx 1 - e^{-bp^r}$$

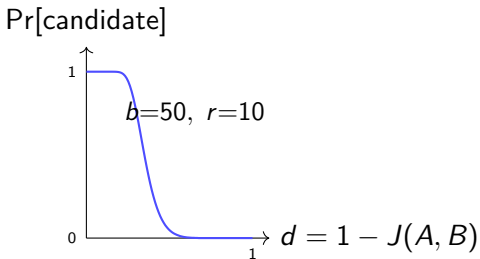
S-Curve

$$f(d) = 1 - (1 - (1 - d)^r)^b$$



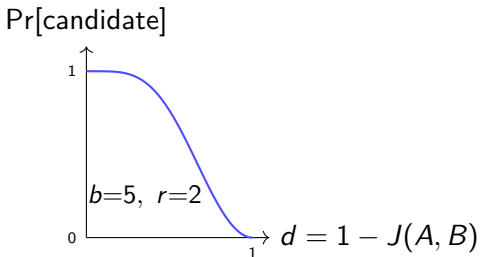
Steep transition: low d pairs likely found; high d pairs mostly filtered.

Steeper Curve: $b=50, r=10$



More bands and rows $\Rightarrow$ sharper step, tighter threshold.

Flatter Curve: $b=5, r=2$



Fewer bands and rows $\Rightarrow$ gradual transition, more false positives and negatives.

Choosing b and r

p^* is the **inflection point**: where $f(p)$ transitions most steeply.

Pairs with $J \gg p^*$ are almost always candidates.

Pairs with $J \ll p^*$ are almost never candidates.

Set b and r to place p^* at your target similarity threshold:

$$p^* \approx \left(\frac{1}{b}\right)^{1/r}$$

b	r	p^*	total hashes
20	5	0.55	100
50	5	0.48	250
10	10	0.79	100

Trade off recall vs. number of false positives by tuning b and r .

LSH Pipeline

1. **Sketch**: compute signature $\text{sig}(A) = (h_1(A), \dots, h_k(A))$ for each set.
2. **Band**: split signature into b bands of r values each.
3. **Bucket**: hash each band into a bucket table. Points sharing a bucket in *any* band are candidates.
4. **Verify**: compute exact similarity for candidate pairs only.

False Positives and False Negatives

For similarity threshold t , with b bands and r rows:

- **False negative** (miss a similar pair, $J \geq t$):

$$\Pr[\text{FN}] = (1 - t^r)^b \approx e^{-bt^r}$$

- **False positive** (flag a dissimilar pair, $J < t$):

$$\Pr[\text{FP}] = 1 - (1 - J^r)^b \approx 1 - e^{-bJ^r}$$

Both decrease as $k = br$ grows; balance them by choosing b and r .

Complexity

- ▶ **Preprocessing:** $O(k \cdot n \cdot |A|)$ to build all signatures.
- ▶ **Query:** $O(k + |\text{candidates}| \cdot |A|)$ per query.
- ▶ **Expected candidates:** $O(n \cdot f(J))$, sublinear for low- J pairs.
- ▶ **Space:** $O(kn)$ for signatures.

Another Setting: Vector Data

So far: sets compared by Jaccard similarity.

New setting: points $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$, compared by **cosine similarity**:

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$$

Examples: document vectors (TF-IDF), word embeddings, user–item ratings.

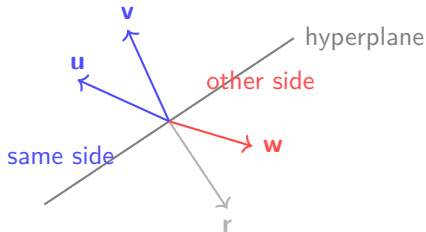
Question: can we build an LSH family for this?

Random Hyperplane LSH

Pick a random vector $\mathbf{r} \in \mathbb{R}^d$ (each entry i.i.d. Gaussian).

Define the hash:

$$h(\mathbf{v}) = \text{sign}(\mathbf{r} \cdot \mathbf{v}) \in \{+1, -1\}$$



Hamming Distance

For binary vectors $\mathbf{u}, \mathbf{v} \in \{0, 1\}^d$, the Hamming distance is:

$d_H(\mathbf{u}, \mathbf{v}) = \text{number of positions where they differ}$

Example: $\mathbf{u} = 10110, \mathbf{v} = 11111 \Rightarrow d_H = 2$

Applications: error-correcting codes, binary feature vectors, DNA sequences.

Bit Sampling LSH

Pick a uniformly random coordinate $i \in \{1, \dots, d\}$ and define:

$$h_i(\mathbf{v}) = v_i$$

Two vectors collide iff they agree on bit i :

$$\Pr[h_i(\mathbf{u}) = h_i(\mathbf{v})] = 1 - \frac{d_H(\mathbf{u}, \mathbf{v})}{d}$$

Close vectors (small d_H) agree on most bits $\Rightarrow$ high collision probability.
Far vectors (large d_H) disagree on many bits $\Rightarrow$ low collision probability.

Why It Works

Each bit position is equally likely to be sampled.

Out of d positions, $\mathbf{u}$ and $\mathbf{v}$ differ on exactly $d_H(\mathbf{u}, \mathbf{v})$ of them.

So the probability of picking a differing position is:

$$\Pr[h_i(\mathbf{u}) \neq h_i(\mathbf{v})] = \frac{d_H(\mathbf{u}, \mathbf{v})}{d}$$

And therefore:

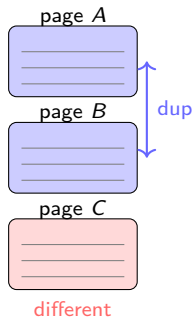
$$\Pr[h_i(\mathbf{u}) = h_i(\mathbf{v})] = 1 - \frac{d_H(\mathbf{u}, \mathbf{v})}{d} \quad \square$$

Near-Duplicate Detection

Problem: find pages with nearly identical content among millions.

- ▶ Represent each page as a set of shingles.
- ▶ Compute MinHash signatures.
- ▶ Pages sharing a bucket are candidate duplicates.

Used by: Google, Bing to deduplicate indices.



Recommendation Systems

Problem: find users similar to a query user among millions.

- ▶ Each user is a binary vector of liked items.
- ▶ LSH finds similar users fast.
- ▶ Recommend items they liked.

Used by: Netflix, Spotify, Amazon.

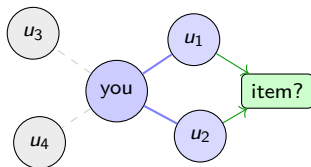
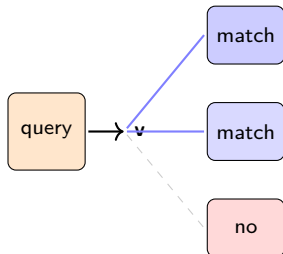


Image and Video Search

Problem: find visually similar images in a large database.

- ▶ Extract CNN embedding $\mathbf{v} \in \mathbb{R}^d$.
- ▶ Random hyperplane LSH buckets similar vectors.
- ▶ Return images in the same bucket.

Used by: reverse image search, face recognition.



Summary

- ▶ LSH maps similar points to the same bucket with high probability.
- ▶ Banding (AND-OR) amplifies the gap between collision probabilities.
- ▶ Tune b and r to control the false positive / false negative trade-off.
- ▶ Works for sets (MinHash), vectors (hyperplane), binary strings (bit sampling).